JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ByteBuffer

java.lang.Object
    java.nio.Buffer
        java.nio.ByteBuffer

All Implemented Interfaces:

`Comparable<ByteBuffer>`

Direct Known Subclasses:

`MappedByteBuffer`

```
public abstract sealed class ByteBuffer
extends Buffer
implements Comparable<ByteBuffer>
permits MappedByteBuffer (not exhaustive)
```

A byte buffer.

This class defines six categories of operations upon byte buffers:

- Absolute and relative *`get`* and *`put`* methods that read and write single bytes;
- Absolute and relative *`bulk get`* methods that transfer contiguous sequences of bytes from this buffer into an array;
- Absolute and relative *`bulk put`* methods that transfer contiguous sequences of bytes from a byte array or some other byte buffer into this buffer;
- Absolute and relative *`get`* and *`put`* methods that read and write values of other primitive types, translating them to and from sequences of bytes in a particular byte order;
- Methods for creating *view buffers*, which allow a byte buffer to be viewed as a buffer containing values of some other primitive type; and
- A method for `compacting` a byte buffer.

Byte buffers can be created either by *`allocation`*, which allocates space for the buffer's content, or by *`wrapping`* an existing byte array into a buffer.

Direct *vs.* non-direct buffers

A byte buffer is either *direct* or *non-direct*. Given a direct byte buffer, the Java virtual machine will make a best effort to perform native I/O operations directly upon it. That is, it will attempt to avoid copying the buffer's content to

(or from) an intermediate buffer before (or after) each invocation of one of the underlying operating system's native I/O operations.

A direct byte buffer may be created by invoking the `allocateDirect` factory method of this class. The buffers returned by this method typically have somewhat higher allocation and deallocation costs than non-direct buffers. The contents of direct buffers may reside outside of the normal garbage-collected heap, and so their impact upon the memory footprint of an application might not be obvious. It is therefore recommended that direct buffers be allocated primarily for large, long-lived buffers that are subject to the underlying system's native I/O operations. In general it is best to allocate direct buffers only when they yield a measurable gain in program performance.

A direct byte buffer may also be created by `mapping` a region of a file directly into memory. An implementation of the Java platform may optionally support the creation of direct byte buffers from native code via JNI. If an instance of one of these kinds of buffers refers to an inaccessible region of memory then an attempt to access that region will not change the buffer's content and will cause an unspecified exception to be thrown either at the time of the access or at some later time.

Whether
a byte buffer is direct or non-direct may be determined by invoking its `isDirect` method. This method is provided so that explicit buffer management can be done in performance-critical code.

Access to binary data

This class defines methods for reading and writing values of all other primitive types, except `boolean`. Primitive values are translated to (or from) sequences of bytes according to the buffer's current byte order, which may be retrieved and modified via the `order` methods. Specific byte orders are represented by instances of the `ByteOrder` class. The initial order of a byte buffer is always `BIG_ENDIAN`.

For access to heterogeneous binary data, that is, sequences of values of different types, this class defines a family of absolute and relative *get* and *put* methods for each type. For 32-bit floating-point values, for example, this class defines:

```
    float      getFloat()
    float      getFloat(int index)
    ByteBuffer putFloat(float f)
    ByteBuffer putFloat(int index, float f)
```

Corresponding methods are defined for the types `char, short, int, long`, and `double`. The index parameters of the absolute *get* and *put* methods are in terms of bytes rather than of the type being read or written.

For access to homogeneous binary data, that is, sequences of values of the same type, this class defines methods that can create *views* of a given byte buffer. A *view buffer* is simply another buffer whose content is backed by the byte buffer. Changes to the byte buffer's content will be visible in the view buffer, and vice versa; the two buffers' position, limit, and mark values are independent. The `asFloatBuffer` method, for example, creates an instance of the `FloatBuffer` class that is backed by the byte buffer upon which the method is invoked. Corresponding view-creation methods are defined for the types `char, short, int, long`, and `double`.

View
buffers have three important advantages over the families of type-specific *get* and *put* methods described above:

- A view buffer is indexed not in terms of bytes but rather in terms of the type-specific size of its values;
- A view buffer provides relative bulk *get* and *put* methods that can transfer contiguous sequences of values between a buffer and an array or some other buffer of the same type; and
- A view buffer is potentially much more efficient because it will be direct if, and only if, its backing byte buffer is direct.

The byte order of a view buffer is fixed to be that of its byte buffer at the time that the view is created.

Invocation chaining

Methods in this class that do not otherwise have a value to return are specified to return the buffer upon which they are invoked. This allows method invocations to be chained. The sequence of statements

```
    bb.putInt(0xCAFEBABE);
    bb.putShort(3);
    bb.putShort(45);
```

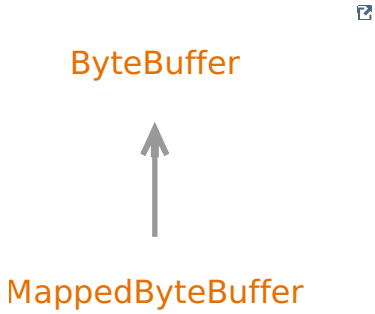
can, for example, be replaced by the single statement

```
    bb.putInt(0xCAFEBABE).putShort(3).putShort(45);
```

Optional operations

Methods specified as *optional operations* throw a ReadOnlyBufferException when invoked on a read-only ByteBuffer. The methods array and arrayOffset throw an UnsupportedOperationException if the ByteBuffer is not backed by an accessible byte array (irrespective of whether the ByteBuffer is read-only).

Sealed Class Hierarchy Graph:



Since:

1.4

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
`final ByteBuffer`	**`alignedSlice`**`(int unitSize)`	Creates a new byte buffer whose content is a shared and aligned subsequence of this buffer's content.
`final int`	**`alignmentOffset`**`(int index, int unitSize)`	Returns the memory address, pointing to the byte at the given index, modulo the given unit size.
`static ByteBuffer`	**`allocate`**`(int capacity)`	Allocates a new byte buffer.
`static ByteBuffer`	**`allocateDirect`**`(int capacity)`	Allocates a new direct byte buffer.
`final byte[]`	**`array`**`()`	Returns the byte array that backs

		this buffer *(optional operation)*.
final int	**arrayOffset**()	Returns the offset within this buffer's backing array of the first element of the buffer *(optional operation)*.
abstract CharBuffer	**asCharBuffer**()	Creates a view of this byte buffer as a char buffer.
abstract DoubleBuffer	**asDoubleBuffer**()	Creates a view of this byte buffer as a double buffer.
abstract FloatBuffer	**asFloatBuffer**()	Creates a view of this byte buffer as a float buffer.
abstract IntBuffer	**asIntBuffer**()	Creates a view of this byte buffer as an int buffer.
abstract LongBuffer	**asLongBuffer**()	Creates a view of this byte buffer as a long buffer.
abstract ByteBuffer	**asReadOnlyBuffer**()	Creates a new, read-only byte buffer that shares this buffer's content.
abstract ShortBuffer	**asShortBuffer**()	Creates a view of this byte buffer as a short buffer.
ByteBuffer	**clear**()	Clears this buffer.
abstract ByteBuffer	**compact**()	Compacts this buffer *(optional operation)*.
int		

	compareTo(ByteBuffer that)	Compares this buffer to another.
abstract ByteBuffer	**duplicate**()	Creates a new byte buffer that shares this buffer's content.
boolean	**equals**(Object ob)	Tells whether or not this buffer is equal to another object.
ByteBuffer	**flip**()	Flips this buffer.
abstract byte	**get**()	Relative *get* method.
ByteBuffer	**get**(byte[] dst)	Relative bulk *get* method.
ByteBuffer	**get**(byte[] dst, int offset, int length)	Relative bulk *get* method.
abstract byte	**get**(int index)	Absolute *get* method.
ByteBuffer	**get**(int index, byte[] dst)	Absolute bulk *get* method.
ByteBuffer	**get**(int index, byte[] dst, int offset, int length)	Absolute bulk *get* method.
abstract char	**getChar**()	Relative *get* method for reading a char value.
abstract char	**getChar**(int index)	Absolute *get* method for reading a char value.
abstract double	**getDouble**()	Relative *get* method for reading a double value.
abstract double	**getDouble**(int index)	Absolute *get* method for reading a double value.
abstract float	**getFloat**()	Relative *get* method for reading a float value.
abstract float	**getFloat**(int index)	Absolute *get* method for reading a float value.

abstract int	**getInt**()	Relative *get* method for reading an int value.
abstract int	**getInt**(int index)	Absolute *get* method for reading an int value.
abstract long	**getLong**()	Relative *get* method for reading a long value.
abstract long	**getLong**(int index)	Absolute *get* method for reading a long value.
abstract short	**getShort**()	Relative *get* method for reading a short value.
abstract short	**getShort**(int index)	Absolute *get* method for reading a short value.
final boolean	**hasArray**()	Tells whether or not this buffer is backed by an accessible byte array.
int	**hashCode**()	Returns the current hash code of this buffer.
abstract boolean	**isDirect**()	Tells whether or not this byte buffer is direct.
ByteBuffer	**limit**(int newLimit)	Sets this buffer's limit.
ByteBuffer	**mark**()	Sets this buffer's mark at its position.
int	**mismatch**(ByteBuffer that)	Finds and returns the relative index of the first mismatch between this buffer and a given buffer.
final ByteOrder	**order**()	Retrieves this buffer's byte order.
final ByteBuffer	**order**(ByteOrder bo)	Modifies this buffer's byte order.
ByteBuffer	**position**(int newPosition)	Sets this buffer's position.
abstract ByteBuffer	**put**(byte b)	

		Relative *put* method *(optional operation)*.
final ByteBuffer	**put**(byte[] src)	Relative bulk *put* method *(optional operation)*.
ByteBuffer	**put**(byte[] src, int offset, int length)	Relative bulk *put* method *(optional operation)*.
abstract ByteBuffer	**put**(int index, byte b)	Absolute *put* method *(optional operation)*.
ByteBuffer	**put**(int index, byte[] src)	Absolute bulk *put* method *(optional operation)*.
ByteBuffer	**put**(int index, byte[] src, int offset, int length)	Absolute bulk *put* method *(optional operation)*.
ByteBuffer	**put**(int index, ByteBuffer src, int offset, int length)	Absolute bulk *put* method *(optional operation)*.
ByteBuffer	**put**(ByteBuffer src)	Relative bulk *put* method *(optional operation)*.
abstract ByteBuffer	**putChar**(char value)	Relative *put* method for writing a char value *(optional operation)*.
abstract ByteBuffer	**putChar**(int index, char value)	Absolute *put* method for writing a char value *(optional operation)*.
abstract ByteBuffer	**putDouble**(double value)	Relative *put* method for writing a double value *(optional operation)*.

abstract ByteBuffer	**putDouble**(int index, double value)	Absolute *put* method for writing a double value *(optional operation)*.
abstract ByteBuffer	**putFloat**(float value)	Relative *put* method for writing a float value *(optional operation)*.
abstract ByteBuffer	**putFloat**(int index, float value)	Absolute *put* method for writing a float value *(optional operation)*.
abstract ByteBuffer	**putInt**(int value)	Relative *put* method for writing an int value *(optional operation)*.
abstract ByteBuffer	**putInt**(int index, int value)	Absolute *put* method for writing an int value *(optional operation)*.
abstract ByteBuffer	**putLong**(int index, long value)	Absolute *put* method for writing a long value *(optional operation)*.
abstract ByteBuffer	**putLong**(long value)	Relative *put* method for writing a long value *(optional operation)*.
abstract ByteBuffer	**putShort**(int index, short value)	Absolute *put* method for writing a short value *(optional operation)*.
abstract ByteBuffer	**putShort**(short value)	Relative *put* method for writing a short value *(optional operation)*.
ByteBuffer	**reset**()	

		Resets this buffer's position to the previously-marked position.
ByteBuffer	**rewind**()	Rewinds this buffer.
abstract ByteBuffer	**slice**()	Creates a new byte buffer whose content is a shared subsequence of this buffer's content.
abstract ByteBuffer	**slice**(int index, int length)	Creates a new byte buffer whose content is a shared subsequence of this buffer's content.
String	**toString**()	Returns a string summarizing the state of this buffer.
static ByteBuffer	**wrap**(byte[] array)	Wraps a byte array into a buffer.
static ByteBuffer	**wrap**(byte[] array, int offset, int length)	Wraps a byte array into a buffer.

Methods declared in class Buffer

capacity, hasRemaining, isReadOnly, limit, position, remaining

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Details

allocateDirect

```
public static ByteBuffer allocateDirect(int capacity)
```

Allocates a new direct byte buffer.

The new buffer's position will be zero, its limit will be its capacity, its mark will be undefined, each of its elements will be initialized to zero, and its byte

order will be BIG_ENDIAN. Whether or not it has a backing array is unspecified.

Parameters:

capacity - The new buffer's capacity, in bytes

Returns:

The new byte buffer

Throws:

IllegalArgumentException - If the capacity is a negative integer

allocate

```
public static ByteBuffer allocate(int capacity)
```

Allocates a new byte buffer.

The new buffer's position will be zero, its limit will be its capacity, its mark will be undefined, each of its elements will be initialized to zero, and its byte order will be BIG_ENDIAN. It will have a backing array, and its array offset will be zero.

Parameters:

capacity - The new buffer's capacity, in bytes

Returns:

The new byte buffer

Throws:

IllegalArgumentException - If the capacity is a negative integer

wrap

```
public static ByteBuffer wrap(byte[] array,
                              int offset,
                              int length)
```

Wraps a byte array into a buffer.

The new buffer will be backed by the given byte array; that is, modifications to the buffer will cause the array to be modified and vice versa. The new buffer's capacity will be array.length, its position will be offset, its limit will be offset + length, its mark will be undefined, and its byte order will be BIG_ENDIAN. Its backing array will be the given array, and its array offset will be zero.

Parameters:

array - The array that will back the new buffer

`offset` - The offset of the subarray to be used; must be non-negative and no larger than `array.length`. The new buffer's position will be set to this value.

`length` - The length of the subarray to be used; must be non-negative and no larger than `array.length - offset`. The new buffer's limit will be set to `offset + length`.

Returns:

The new byte buffer

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `offset` and `length` parameters do not hold

wrap

```
public static ByteBuffer wrap(byte[] array)
```

Wraps a byte array into a buffer.

The new buffer will be backed by the given byte array; that is, modifications to the buffer will cause the array to be modified and vice versa. The new buffer's capacity and limit will be `array.length`, its position will be zero, its mark will be undefined, and its byte order will be `BIG_ENDIAN`. Its `backing array` will be the given array, and its `array offset` will be zero.

Parameters:

`array` - The array that will back this buffer

Returns:

The new byte buffer

slice

```
public abstract ByteBuffer slice()
```

Creates a new byte buffer whose content is a shared subsequence of this buffer's content.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer, its mark will be undefined, and its byte order will be `BIG_ENDIAN`. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Specified by:

slice in class Buffer

Returns:

The new byte buffer

See Also:

alignedSlice(int)

slice

```
public abstract ByteBuffer slice(int index,
                                 int length)
```

Creates a new byte buffer whose content is a shared subsequence of this buffer's content.

The content of the new buffer will start at position index in this buffer, and will contain length elements. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be length, its mark will be undefined, and its byte order will be BIG_ENDIAN. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Specified by:

slice in class Buffer

Parameters:

index - The position in this buffer at which the content of the new buffer will start; must be non-negative and no larger than limit()

length - The number of elements the new buffer will contain; must be non-negative and no larger than limit() - index

Returns:

The new buffer

Throws:

IndexOutOfBoundsException - If index is negative or greater than limit(), length is negative, or length > limit() - index

Since:

13

duplicate

```
public abstract ByteBuffer duplicate()
```

Creates a new byte buffer that shares this buffer's content.

The content of the new buffer will be that of this buffer. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's capacity, limit, position, and mark values will be identical to those of this buffer, and its byte order will be `BIG_ENDIAN`. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Specified by:

`duplicate` in class `Buffer`

Returns:

The new byte buffer

asReadOnlyBuffer

```
public abstract ByteBuffer asReadOnlyBuffer()
```

Creates a new, read-only byte buffer that shares this buffer's content.

The content of the new buffer will be that of this buffer. Changes to this buffer's content will be visible in the new buffer; the new buffer itself, however, will be read-only and will not allow the shared content to be modified. The two buffers' position, limit, and mark values will be independent.

The new buffer's capacity, limit, position, and mark values will be identical to those of this buffer, and its byte order will be `BIG_ENDIAN`.

If this buffer is itself read-only then this method behaves in exactly the same way as the `duplicate` method.

Returns:

The new, read-only byte buffer

get

```
public abstract byte get()
```

Relative *get* method. Reads the byte at this buffer's current position, and then increments the position.

Returns:

The byte at the buffer's current position

Throws:

`BufferUnderflowException` - If the buffer's current position is not smaller than its limit

put

```
public abstract ByteBuffer put(byte b)
```

Relative *put* method *(optional operation)*.

Writes the given byte into this buffer at the current position, and then increments the position.

Parameters:

`b` - The byte to be written

Returns:

This buffer

Throws:

`BufferOverflowException` - If this buffer's current position is not smaller than its limit

`ReadOnlyBufferException` - If this buffer is read-only

get

```
public abstract byte get(int index)
```

Absolute *get* method. Reads the byte at the given index.

Parameters:

`index` - The index from which the byte will be read

Returns:

The byte at the given index

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit

put

```
public abstract ByteBuffer put(int index,
                               byte b)
```

Absolute *put* method *(optional operation)*.

Writes the given byte into this buffer at the given index.

Parameters:

`index` - The index at which the byte will be written

`b` - The byte value to be written

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit

`ReadOnlyBufferException` - If this buffer is read-only

get

```
public ByteBuffer get(byte[] dst,
                      int offset,
                      int length)
```

Relative bulk *get* method.

This method transfers bytes from this buffer into the given destination array. If there are fewer bytes remaining in the buffer than are required to satisfy the request, that is, if `length > remaining()`, then no bytes are transferred and a `BufferUnderflowException` is thrown.

Otherwise, this method copies `length` bytes from this buffer into the given array, starting at the current position of this buffer and at the given offset in the array. The position of this buffer is then incremented by `length`.

In other words, an invocation of this method of the form `src.get(dst, off, len)` has exactly the same effect as the loop

```
    for (int i = off; i < off + len; i++)
        dst[i] = src.get();
```

except that it first checks that there are sufficient bytes in this buffer and it is potentially much more efficient.

Parameters:

`dst` - The array into which bytes are to be written

`offset` - The offset within the array of the first byte to be written; must be non-negative and no larger than `dst.length`

`length` - The maximum number of bytes to be written to the given array; must be non-negative and no larger than `dst.length - offset`

Returns:

This buffer

Throws:

`BufferUnderflowException` - If there are fewer than `length` bytes remaining in this buffer

`IndexOutOfBoundsException` - If the preconditions on the `offset` and `length` parameters do not hold

get

```
public ByteBuffer get(byte[] dst)
```

Relative bulk *get* method.

This method transfers bytes from this buffer into the given destination array. An invocation of this method of the form `src.get(a)` behaves in exactly the same way as the invocation

```
    src.get(a, 0, a.length)
```

Parameters:

`dst` - The destination array

Returns:

This buffer

Throws:

`BufferUnderflowException` - If there are fewer than `length` bytes remaining in this buffer

get

```
public ByteBuffer get(int index,
                      byte[] dst,
                      int offset,
                      int length)
```

Absolute bulk *get* method.

This method transfers `length` bytes from this buffer into the given array, starting at the given index in this buffer and at the given offset in the array. The position of this buffer is unchanged.

An invocation of this method of the form `src.get(index, dst, offset, length)` has exactly the same effect as the following loop except that it first checks the consistency of the supplied parameters and it is potentially much more efficient:

```
    for (int i = offset, j = index; i < offset + length; i++, j++)
        dst[i] = src.get(j);
```

Parameters:

`index` - The index in this buffer from which the first byte will be read; must be non-negative and less than `limit()`

`dst` - The destination array

`offset` - The offset within the array of the first byte to be written; must be non-negative and less than `dst.length`

`length` - The number of bytes to be written to the given array; must be non-negative and no larger than the smaller of `limit() - index` and `dst.length - offset`

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `index`, `offset`, and `length` parameters do not hold

Since:

13

get

```
public ByteBuffer get(int index,
                      byte[] dst)
```

Absolute bulk *get* method.

This method transfers bytes from this buffer into the given destination array. The position of this buffer is unchanged. An invocation of this method of the form `src.get(index, dst)` behaves in exactly the same way as the invocation:

```
    src.get(index, dst, 0, dst.length)
```

Parameters:

`index` - The index in this buffer from which the first byte will be read; must be non-negative and less than `limit()`

`dst` - The destination array

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative, not smaller than `limit()`, or `limit() - index < dst.length`

Since:

13

put

`public ByteBuffer put(ByteBuffer src)`

Relative bulk *put* method *(optional operation)*.

This method transfers the bytes remaining in the given source buffer into this buffer. If there are more bytes remaining in the source buffer than in this buffer, that is, if `src.remaining() > remaining()`, then no bytes are transferred and a `BufferOverflowException` is thrown.

Otherwise, this method copies *n* = `src.remaining()` bytes from the given buffer into this buffer, starting at each buffer's current position. The positions of both buffers are then incremented by *n*.

In other words, an invocation of this method of the form `dst.put(src)` has exactly the same effect as the loop

```
    while (src.hasRemaining())
        dst.put(src.get());
```

except that it first checks that there is sufficient space in this buffer and it is potentially much more efficient. If this buffer and the source buffer share the same backing array or memory, then the result will be as if the source elements were first copied to an intermediate location before being written into this buffer.

Parameters:

`src` - The source buffer from which bytes are to be read; must not be this buffer

Returns:

This buffer

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer for the remaining bytes in the source buffer

`IllegalArgumentException` - If the source buffer is this buffer

`ReadOnlyBufferException` - If this buffer is read-only

put

```
public ByteBuffer put(int index,
                      ByteBuffer src,
                      int offset,
                      int length)
```

Absolute bulk *put* method *(optional operation)*.

This method transfers `length` bytes into this buffer from the given source buffer, starting at the given `offset` in the source buffer and the given `index` in this buffer. The positions of both buffers are unchanged.

In other words, an invocation of this method of the form `dst.put(index, src, offset, length)` has exactly the same effect as the loop

```
    for (int i = offset, j = index; i < offset + length; i++, j++)
        dst.put(j, src.get(i));
```

except that it first checks the consistency of the supplied parameters and it is potentially much more efficient. If this buffer and the source buffer share the same backing array or memory, then the result will be as if the source elements were first copied to an intermediate location before being written into this buffer.

Parameters:

`index` - The index in this buffer at which the first byte will be written; must be non-negative and less than `limit()`

`src` - The buffer from which bytes are to be read

`offset` - The index within the source buffer of the first byte to be read; must be non-negative and less than `src.limit()`

`length` - The number of bytes to be read from the given buffer; must be non-negative and no larger than the smaller of `limit() - index` and `src.limit() - offset`

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `index`, `offset`, and `length` parameters do not hold

`ReadOnlyBufferException` - If this buffer is read-only

Since:

put

```
public ByteBuffer put(byte[] src,
                      int offset,
                      int length)
```

Relative bulk *put* method *(optional operation)*.

This method transfers bytes into this buffer from the given source array. If there are more bytes to be copied from the array than remain in this buffer, that is, if `length > remaining()`, then no bytes are transferred and a `BufferOverflowException` is thrown.

Otherwise, this method copies `length` bytes from the given array into this buffer, starting at the given offset in the array and at the current position of this buffer. The position of this buffer is then incremented by `length`.

In other words, an invocation of this method of the form `dst.put(src, off, len)` has exactly the same effect as the loop

```
    for (int i = off; i < off + len; i++)
        dst.put(src[i]);
```

except that it first checks that there is sufficient space in this buffer and it is potentially much more efficient.

Parameters:

`src` - The array from which bytes are to be read

`offset` - The offset within the array of the first byte to be read; must be non-negative and no larger than `src.length`

`length` - The number of bytes to be read from the given array; must be non-negative and no larger than `src.length - offset`

Returns:

This buffer

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer

`IndexOutOfBoundsException` - If the preconditions on the `offset` and `length` parameters do not hold

`ReadOnlyBufferException` - If this buffer is read-only

put

```
public final ByteBuffer put(byte[] src)
```

Relative bulk *put* method *(optional operation)*.

This method transfers the entire content of the given source byte array into this buffer. An invocation of this method of the form `dst.put(a)` behaves in exactly the same way as the invocation

```
    dst.put(a, 0, a.length)
```

Parameters:

`src` - The source array

Returns:

This buffer

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

put

```
public ByteBuffer put(int index,
                      byte[] src,
                      int offset,
                      int length)
```

Absolute bulk *put* method *(optional operation)*.

This method transfers `length` bytes from the given array, starting at the given offset in the array and at the given index in this buffer. The position of this buffer is unchanged.

An invocation of this method of the form `dst.put(index, src, offset, length)` has exactly the same effect as the following loop except that it first checks the consistency of the supplied parameters and it is potentially much more efficient:

```
    for (int i = offset, j = index; i < offset + length; i++, j++)
        dst.put(j, src[i]);
```

Parameters:

`index` - The index in this buffer at which the first byte will be written; must be non-negative and less than `limit()`

`src` - The array from which bytes are to be read

`offset` - The offset within the array of the first byte to be read; must be non-negative and less than `src.length`

`length` - The number of bytes to be read from the given array; must be non-negative and no larger than the smaller of `limit() - index` and `src.length - offset`

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `index`, `offset`, and `length` parameters do not hold

`ReadOnlyBufferException` - If this buffer is read-only

Since:

13

put

```
public ByteBuffer put(int index,
                      byte[] src)
```

Absolute bulk *put* method *(optional operation)*.

This method copies bytes into this buffer from the given source array. The position of this buffer is unchanged. An invocation of this method of the form `dst.put(index, src)` behaves in exactly the same way as the invocation:

```
    dst.put(index, src, 0, src.length);
```

Parameters:

`index` - The index in this buffer at which the first byte will be written; must be non-negative and less than `limit()`

`src` - The array from which bytes are to be read

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative, not smaller than `limit()`, or `limit() - index < src.length`

`ReadOnlyBufferException` - If this buffer is read-only

Since:

13

hasArray

```
public final boolean hasArray()
```

Tells whether or not this buffer is backed by an accessible byte array.

If this method returns `true` then the `array` and `arrayOffset` methods may safely be invoked.

Specified by:

`hasArray` in class `Buffer`

Returns:

`true` if, and only if, this buffer is backed by an array and is not read-only

array

```
public final byte[] array()
```

Returns the byte array that backs this buffer *(optional operation)*.

Modifications to this buffer's content will cause the returned array's content to be modified, and vice versa.

Invoke the `hasArray` method before invoking this method in order to ensure that this buffer has an accessible backing array.

Specified by:

`array` in class `Buffer`

Returns:

The array that backs this buffer

Throws:

`ReadOnlyBufferException` - If this buffer is backed by an array but is read-only

`UnsupportedOperationException` - If this buffer is not backed by an accessible array

arrayOffset

```
public final int arrayOffset()
```

Returns the offset within this buffer's backing array of the first element of the buffer *(optional operation)*.

If this buffer is backed by an array then buffer position *p* corresponds to array index *p* + `arrayOffset()`.

Invoke the hasArray method before invoking this method in order to ensure that this buffer has an accessible backing array.

Specified by:

arrayOffset in class Buffer

Returns:

The offset within this buffer's array of the first element of the buffer

Throws:

ReadOnlyBufferException - If this buffer is backed by an array but is read-only

UnsupportedOperationException - If this buffer is not backed by an accessible array

position

```
public ByteBuffer position(int newPosition)
```

Sets this buffer's position. If the mark is defined and larger than the new position then it is discarded.

Overrides:

position in class Buffer

Parameters:

newPosition - The new position value; must be non-negative and no larger than the current limit

Returns:

This buffer

Since:

9

limit

```
public ByteBuffer limit(int newLimit)
```

Sets this buffer's limit. If the position is larger than the new limit then it is set to the new limit. If the mark is defined and larger than the new limit then it is discarded.

Overrides:

limit in class Buffer

Parameters:

newLimit - The new limit value; must be non-negative and no larger than this buffer's capacity

Returns:

This buffer

Since:

9

mark

```
public ByteBuffer mark()
```

Sets this buffer's mark at its position.

Overrides:

`mark` in class `Buffer`

Returns:

This buffer

Since:

9

reset

```
public ByteBuffer reset()
```

Resets this buffer's position to the previously-marked position.

Invoking this method neither changes nor discards the mark's value.

Overrides:

`reset` in class `Buffer`

Returns:

This buffer

Since:

9

clear

```
public ByteBuffer clear()
```

Clears this buffer. The position is set to zero, the limit is set to the capacity, and the mark is discarded.

Invoke this method before using a sequence of channel-read or *put* operations to fill this buffer. For example:

```
    buf.clear();     // Prepare buffer for reading
    in.read(buf);    // Read data
```

This method does not actually erase the data in the buffer, but it is named as if it did because it will most often be used in situations in which that might as well be the case.

Overrides:

`clear` in class `Buffer`

Returns:

This buffer

Since:

9

flip

`public ByteBuffer flip()`

Flips this buffer. The limit is set to the current position and then the position is set to zero. If the mark is defined then it is discarded.

After a sequence of channel-read or *put* operations, invoke this method to prepare for a sequence of channel-write or relative *get* operations. For example:

```
    buf.put(magic);    // Prepend header
    in.read(buf);      // Read data into rest of buffer
    buf.flip();        // Flip buffer
    out.write(buf);    // Write header + data to channel
```

This method is often used in conjunction with the `compact` method when transferring data from one place to another.

Overrides:

`flip` in class `Buffer`

Returns:

This buffer

Since:

9

rewind

`public ByteBuffer rewind()`

Rewinds this buffer. The position is set to zero and the mark is discarded.

Invoke this method before a sequence of channel-write or *get* operations, assuming that the limit has already been set appropriately. For example:

```
    out.write(buf);    // Write remaining data
    buf.rewind();      // Rewind buffer
    buf.get(array);    // Copy data into array
```

Overrides:

`rewind` in class `Buffer`

Returns:

This buffer

Since:

9

compact

```
public abstract ByteBuffer compact()
```

Compacts this buffer *(optional operation)*.

The bytes between the buffer's current position and its limit, if any, are copied to the beginning of the buffer. That is, the byte at index p = `position()` is copied to index zero, the byte at index p + 1 is copied to index one, and so forth until the byte at index `limit()` - 1 is copied to index n = `limit()` - 1 - p. The buffer's position is then set to $n+1$ and its limit is set to its capacity. The mark, if defined, is discarded.

The buffer's position is set to the number of bytes copied, rather than to zero, so that an invocation of this method can be followed immediately by an invocation of another relative *put* method.

Invoke this method after writing data from a buffer in case the write was incomplete. The following loop, for example, copies bytes from one channel to another via the buffer `buf`:

```
    buf.clear();          // Prepare buffer for use
    while (in.read(buf) >= 0 || buf.position != 0) {
        buf.flip();
        out.write(buf);
        buf.compact();    // In case of partial write
    }
```

Returns:

This buffer

Throws:

`ReadOnlyBufferException` - If this buffer is read-only

isDirect

```
public abstract boolean isDirect()
```

Tells whether or not this byte buffer is direct.

Specified by:

`isDirect` in class `Buffer`

Returns:

`true` if, and only if, this buffer is direct

toString

```
public String toString()
```

Returns a string summarizing the state of this buffer.

Overrides:

`toString` in class `Object`

Returns:

A summary string

hashCode

```
public int hashCode()
```

Returns the current hash code of this buffer.

The hash code of a byte buffer depends only upon its remaining elements; that is, upon the elements from `position()` up to, and including, the element at `limit() - 1`.

Because buffer hash codes are content-dependent, it is inadvisable to use buffers as keys in hash maps or similar data structures unless it is known that their contents will not change.

Overrides:

`hashCode` in class `Object`

Returns:

The current hash code of this buffer

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

equals

```
public boolean equals(Object ob)
```

Tells whether or not this buffer is equal to another object.

Two byte buffers are equal if, and only if,

1. They have the same element type,
2. They have the same number of remaining elements, and
3. The two sequences of remaining elements, considered independently of their starting positions, are pointwise equal.

A byte buffer is not equal to any other type of object.

Overrides:

`equals` in class `Object`

Parameters:

`ob` - The object to which this buffer is to be compared

Returns:

`true` if, and only if, this buffer is equal to the given object

See Also:

`Object.hashCode()`, `HashMap`

compareTo

```
public int compareTo(ByteBuffer that)
```

Compares this buffer to another.

Two byte buffers are compared by comparing their sequences of remaining elements lexicographically, without regard to the starting position of each sequence within its corresponding buffer. Pairs of `byte` elements are compared as if by invoking `Byte.compare(byte,byte)`.

A byte buffer is not comparable to any other type of object.

Specified by:

`compareTo` in interface `Comparable<ByteBuffer>`

Parameters:

`that` - the object to be compared.

Returns:

A negative integer, zero, or a positive integer as this buffer is less than, equal to, or greater than the given buffer

mismatch

```
public int mismatch(ByteBuffer that)
```

Finds and returns the relative index of the first mismatch between this buffer and a given buffer. The index is relative to the `position` of each buffer and will be in the range of 0 (inclusive) up to the smaller of the `remaining` elements in each buffer (exclusive).

If the two buffers share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two buffers at that index within the respective buffers. If one buffer is a proper prefix of the other then the returned index is the smaller of the remaining elements in each buffer, and it follows that the index is only valid for the buffer with the larger number of remaining elements. Otherwise, there is no mismatch.

Parameters:

`that` - The byte buffer to be tested for a mismatch with this buffer

Returns:

The relative index of the first mismatch between this and the given buffer, otherwise -1 if no mismatch.

Since:

11

order

```
public final ByteOrder order()
```

Retrieves this buffer's byte order.

The byte order is used when reading or writing multibyte values, and when creating buffers that are views of this byte buffer. The order of a newly-created byte buffer is always `BIG_ENDIAN`.

Returns:

This buffer's byte order

order

```
public final ByteBuffer order(ByteOrder bo)
```

Modifies this buffer's byte order.

Parameters:

`bo` - The new byte order, either `BIG_ENDIAN` or `LITTLE_ENDIAN`

Returns:

This buffer

alignmentOffset

```
public final int alignmentOffset(int index,
                                 int unitSize)
```

Returns the memory address, pointing to the byte at the given index, modulo the given unit size.

The return value is non-negative in the range of `0` (inclusive) up to `unitSize` (exclusive), with zero indicating that the address of the byte at the index is aligned for the unit size, and a positive value that the address is misaligned for the unit size. If the address of the byte at the index is misaligned, the return value represents how much the index should be adjusted to locate a byte at an aligned address. Specifically, the index should either be decremented by the return value if the latter is not greater than `index`, or be incremented by the unit size minus the return value. Therefore given

```
    int value = alignmentOffset(index, unitSize)
```

then the identities

```
    alignmentOffset(index - value, unitSize) == 0, value <= index
```

and

```
    alignmentOffset(index + (unitSize - value), unitSize) == 0
```

must hold.

API Note:

This method may be utilized to determine if unit size bytes from an index can be accessed atomically, if supported by the native platform.

Parameters:

`index` - The index to query for alignment offset, must be non-negative, no upper bounds check is performed

`unitSize` - The unit size in bytes, must be a power of 2

Returns:

The indexed byte's memory address modulo the unit size

Throws:

`IllegalArgumentException` - If the index is negative or the unit size is not a power of 2

`UnsupportedOperationException` - If the buffer is non-direct, and `unitSize > 1`

Since:

9

See Also:

`alignedSlice(int)`

alignedSlice

```
public final ByteBuffer alignedSlice(int unitSize)
```

Creates a new byte buffer whose content is a shared and aligned subsequence of this buffer's content.

The content of the new buffer will start at this buffer's current position rounded up to the index of the nearest aligned byte for the given unit size, and end at this buffer's limit rounded down to the index of the nearest aligned byte for the given unit size. If rounding results in out-of-bound values then the new buffer's capacity and limit will be zero. If rounding is within bounds the following expressions will be true for a new buffer `nb` and unit size `unitSize`:

```
    nb.alignmentOffset(0, unitSize) == 0
    nb.alignmentOffset(nb.limit(), unitSize) == 0
```

Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer or fewer subject to alignment, its mark will be undefined, and its byte order will be `BIG_ENDIAN`. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

API Note:

This method may be utilized to create a new buffer where unit size bytes from index, that is a multiple of the unit size, may be accessed atomically, if supported by the native platform.

Parameters:

`unitSize` - The unit size in bytes, must be a power of 2

Returns:

The new byte buffer

Throws:

`IllegalArgumentException` - If the unit size not a power of 2

`UnsupportedOperationException` - If the buffer is non-direct, and `unitSize > 1`

Since:

9

See Also:

`alignmentOffset(int, int)`,
`slice()`

getChar

```
public abstract char getChar()
```

Relative *get* method for reading a char value.

Reads the next two bytes at this buffer's current position, composing them into a char value according to the current byte order, and then increments the position by two.

Returns:

The char value at the buffer's current position

Throws:

`BufferUnderflowException` - If there are fewer than two bytes remaining in this buffer

putChar

```
public abstract ByteBuffer putChar(char value)
```

Relative *put* method for writing a char value *(optional operation)*.

Writes two bytes containing the given char value, in the current byte order, into this buffer at the current position, and then increments the position by two.

Parameters:

`value` - The char value to be written

Returns:

This buffer

Throws:

`BufferOverflowException` - If there are fewer than two bytes remaining in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

getChar

```
public abstract char getChar(int index)
```

Absolute *get* method for reading a char value.

Reads two bytes at the given index, composing them into a char value according to the current byte order.

Parameters:

`index` - The index from which the bytes will be read

Returns:

The char value at the given index

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus one

putChar

```
public abstract ByteBuffer putChar(int index,
                                   char value)
```

Absolute *put* method for writing a char value *(optional operation)*.

Writes two bytes containing the given char value, in the current byte order, into this buffer at the given index.

Parameters:

`index` - The index at which the bytes will be written

`value` - The char value to be written

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus one

`ReadOnlyBufferException` - If this buffer is read-only

asCharBuffer

```
public abstract CharBuffer asCharBuffer()
```

Creates a view of this byte buffer as a char buffer.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer divided by two, its mark will be undefined, and its byte order will be that of the byte buffer at the moment the view is created. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Returns:

A new char buffer

getShort

```
public abstract short getShort()
```

Relative *get* method for reading a short value.

Reads the next two bytes at this buffer's current position, composing them into a short value according to the current byte order, and then increments the position by two.

Returns:

The short value at the buffer's current position

Throws:

`BufferUnderflowException` - If there are fewer than two bytes remaining in this buffer

putShort

```
public abstract ByteBuffer putShort(short value)
```

Relative *put* method for writing a short value *(optional operation)*.

Writes two bytes containing the given short value, in the current byte order, into this buffer at the current position, and then increments the position by two.

Parameters:

`value` - The short value to be written

Returns:

This buffer

Throws:

`BufferOverflowException` - If there are fewer than two bytes remaining in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

getShort

```
public abstract short getShort(int index)
```

Absolute *get* method for reading a short value.

Reads two bytes at the given index, composing them into a short value according to the current byte order.

Parameters:

`index` - The index from which the bytes will be read

Returns:

The short value at the given index

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus one

putShort

```
public abstract ByteBuffer putShort(int index,
                                    short value)
```

Absolute *put* method for writing a short value *(optional operation)*.

Writes two bytes containing the given short value, in the current byte order, into this buffer at the given index.

Parameters:

`index` - The index at which the bytes will be written

`value` - The short value to be written

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus one

`ReadOnlyBufferException` - If this buffer is read-only

asShortBuffer

```
public abstract ShortBuffer asShortBuffer()
```

Creates a view of this byte buffer as a short buffer.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer divided by two, its mark will be undefined, and its byte order will be that of the byte buffer at the moment the view is created. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Returns:

A new short buffer

getInt

```
public abstract int getInt()
```

Relative *get* method for reading an int value.

Reads the next four bytes at this buffer's current position, composing them into an int value according to the current byte order, and then increments the position by four.

Returns:

The int value at the buffer's current position

Throws:

`BufferUnderflowException` - If there are fewer than four bytes remaining in this buffer

putInt

```
public abstract ByteBuffer putInt(int value)
```

Relative *put* method for writing an int value *(optional operation)*.

Writes four bytes containing the given int value, in the current byte order, into this buffer at the current position, and then increments the position by four.

Parameters:

`value` - The int value to be written

Returns:

This buffer

Throws:

`BufferOverflowException` - If there are fewer than four bytes remaining in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

getInt

```
public abstract int getInt(int index)
```

Absolute *get* method for reading an int value.

Reads four bytes at the given index, composing them into a int value according to the current byte order.

Parameters:

`index` - The index from which the bytes will be read

Returns:

The int value at the given index

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus three

putInt

```
public abstract ByteBuffer putInt(int index,
                                  int value)
```

Absolute *put* method for writing an int value *(optional operation)*.

Writes four bytes containing the given int value, in the current byte order, into this buffer at the given index.

Parameters:

`index` - The index at which the bytes will be written

`value` - The int value to be written

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus three

`ReadOnlyBufferException` - If this buffer is read-only

asIntBuffer

```
public abstract IntBuffer asIntBuffer()
```

Creates a view of this byte buffer as an int buffer.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer divided by four, its mark will be undefined, and its byte order will be that of the byte buffer at the moment the view is created. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Returns:

A new int buffer

getLong

```
public abstract long getLong()
```

Relative *get* method for reading a long value.

Reads the next eight bytes at this buffer's current position, composing them into a long value according to the current byte order, and then increments the position by eight.

Returns:

The long value at the buffer's current position

Throws:

`BufferUnderflowException` - If there are fewer than eight bytes remaining in this buffer

putLong

```
public abstract ByteBuffer putLong(long value)
```

Relative *put* method for writing a long value *(optional operation)*.

Writes eight bytes containing the given long value, in the current byte order, into this buffer at the current position, and then increments the position by eight.

Parameters:

`value` - The long value to be written

Returns:

This buffer

Throws:

BufferOverflowException - If there are fewer than eight bytes remaining in this buffer

ReadOnlyBufferException - If this buffer is read-only

getLong

```
public abstract long getLong(int index)
```

Absolute *get* method for reading a long value.

Reads eight bytes at the given index, composing them into a long value according to the current byte order.

Parameters:

index - The index from which the bytes will be read

Returns:

The long value at the given index

Throws:

IndexOutOfBoundsException - If index is negative or not smaller than the buffer's limit, minus seven

putLong

```
public abstract ByteBuffer putLong(int index,
                                   long value)
```

Absolute *put* method for writing a long value *(optional operation)*.

Writes eight bytes containing the given long value, in the current byte order, into this buffer at the given index.

Parameters:

index - The index at which the bytes will be written

value - The long value to be written

Returns:

This buffer

Throws:

IndexOutOfBoundsException - If index is negative or not smaller than the buffer's limit, minus seven

ReadOnlyBufferException - If this buffer is read-only

asLongBuffer

```
public abstract LongBuffer asLongBuffer()
```

Creates a view of this byte buffer as a long buffer.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer divided by eight, its mark will be undefined, and its byte order will be that of the byte buffer at the moment the view is created. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Returns:

A new long buffer

getFloat

```
public abstract float getFloat()
```

Relative *get* method for reading a float value.

Reads the next four bytes at this buffer's current position, composing them into a float value according to the current byte order, and then increments the position by four.

Returns:

The float value at the buffer's current position

Throws:

`BufferUnderflowException` - If there are fewer than four bytes remaining in this buffer

putFloat

```
public abstract ByteBuffer putFloat(float value)
```

Relative *put* method for writing a float value *(optional operation)*.

Writes four bytes containing the given float value, in the current byte order, into this buffer at the current position, and then increments the position by four.

Parameters:

`value` - The float value to be written

Returns:

This buffer

Throws:

`BufferOverflowException` - If there are fewer than four bytes remaining in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

getFloat

```
public abstract float getFloat(int index)
```

Absolute *get* method for reading a float value.

Reads four bytes at the given index, composing them into a float value according to the current byte order.

Parameters:

`index` - The index from which the bytes will be read

Returns:

The float value at the given index

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus three

putFloat

```
public abstract ByteBuffer putFloat(int index,
                                    float value)
```

Absolute *put* method for writing a float value *(optional operation)*.

Writes four bytes containing the given float value, in the current byte order, into this buffer at the given index.

Parameters:

`index` - The index at which the bytes will be written

`value` - The float value to be written

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus three

`ReadOnlyBufferException` - If this buffer is read-only

asFloatBuffer

```
public abstract FloatBuffer asFloatBuffer()
```

Creates a view of this byte buffer as a float buffer.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer divided by four, its mark will be undefined, and its byte order will be that of the byte buffer at the moment the view is created. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Returns:

A new float buffer

getDouble

```
public abstract double getDouble()
```

Relative *get* method for reading a double value.

Reads the next eight bytes at this buffer's current position, composing them into a double value according to the current byte order, and then increments the position by eight.

Returns:

The double value at the buffer's current position

Throws:

`BufferUnderflowException` - If there are fewer than eight bytes remaining in this buffer

putDouble

```
public abstract ByteBuffer putDouble(double value)
```

Relative *put* method for writing a double value *(optional operation)*.

Writes eight bytes containing the given double value, in the current byte order, into this buffer at the current position, and then increments the position by eight.

Parameters:

`value` - The double value to be written

Returns:

This buffer

Throws:

`BufferOverflowException` - If there are fewer than eight bytes remaining in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

getDouble

```
public abstract double getDouble(int index)
```

Absolute *get* method for reading a double value.

Reads eight bytes at the given index, composing them into a double value according to the current byte order.

Parameters:

`index` - The index from which the bytes will be read

Returns:

The double value at the given index

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus seven

putDouble

```
public abstract ByteBuffer putDouble(int index,
                                     double value)
```

Absolute *put* method for writing a double value *(optional operation)*.

Writes eight bytes containing the given double value, in the current byte order, into this buffer at the given index.

Parameters:

`index` - The index at which the bytes will be written

`value` - The double value to be written

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit, minus seven

`ReadOnlyBufferException` - If this buffer is read-only

asDoubleBuffer

```
public abstract DoubleBuffer asDoubleBuffer()
```

Creates a view of this byte buffer as a double buffer.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer divided by eight, its mark will be undefined, and its byte order will be that of the byte buffer at the moment the view is created. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Returns:

A new double buffer

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.